Trabalho Prático | DGT2811 Desenvolvimento Back-End Corporativo com Java E Cloud

Gleice Ferreira Lordeiro - Matrícula: 202406131669

***Campus/Polo**: Rua Oscar Soares, 1466 - California - Nova Iguaçu - RJ - CEP.: 26.220-099*

Desenvolvimento Back-End Corporativo com Java E Cloud – Número da Turma – 2025.4 FLEX

Objetivo da Prática

O objetivo desta prática foi desenvolver uma aplicação corporativa Java EE utilizando JPA para persistência de dados, EJB para regras de negócio, Servlets e JSP para camada Web, organizando o projeto em camadas seguindo o padrão MVC. O projeto foi executado no NetBeans com GlassFish 6 e banco de dados SQL Server.

1º Procedimento | Camadas de Persistência e Controle

Cliente.java (Entidade JPA):

```
package cadastroee.model;

import jakarta.persistence.Entity;

import jakarta.persistence.Id;

import jakarta.persistence.GeneratedValue;

import jakarta.persistence.GenerationType;

import jakarta.persistence.Column;

import java.io.Serializable;

@Entity

public class Cliente implements Serializable {

    private static final long serialVersionUID = 1L;

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    @Column(name = "id_cliente")

    private Long id;

    @Column(nullable = false, length = 100)

    private String nome;
```

```
    @Column(length = 150)
    private String email;
    public Cliente() {
    }
    public Cliente(String nome, String email) {
        this.nome = nome;
        this.email = email;
    }
    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }
    public String getNome() {
        return nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
}
```

ClienteFacade.java (EJB):

```
package cadastroee.controller;
import cadastroee.facade.ClienteFacadeLocal;
import cadastroee.model.Cliente;
import jakarta.ejb.Stateless;
import jakarta.persistence.EntityManager;
```

```
import jakarta.persistence.PersistenceContext;
import java.util.List;
@Stateless
public class ClienteFacade implements ClienteFacadeLocal {
    @PersistenceContext(unitName = "CadastroEE-ejbPU")
    private EntityManager em;
    @Override
    public List<Cliente> findAll() {
        return em.createQuery("SELECT c FROM Cliente c", Cliente.class)
                 .getResultList();
    }
}
```

Produto.java

```java
package cadastroee.model;

import jakarta.persistence.*;
import java.io.Serializable;
import java.math.BigDecimal;

@Entity
@Table(name = "produto")
public class Produto implements Serializable {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_produto")
    private Integer id;

    @Column(nullable = false, length = 100)
    private String nome;

    @Column(name = "estoque", nullable = false)
    private Integer quantidade;

    @Column(nullable = false)
    private BigDecimal preco;

    public Produto() {
    }

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
```

```
        this.nome = nome;
    }
    public Integer getQuantidade() {
        return quantidade;
    }
    public void setQuantidade(Integer quantidade) {
        this.quantidade = quantidade;
    }
    public BigDecimal getPreco() {
        return preco;
    }
    public void setPreco(BigDecimal preco) {
        this.preco = preco;
    }
}
```

ProdutoFacade.java

```
package cadastroee.facade;

import cadastroee.model.Produto;
import jakarta.ejb.Stateless;
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;
import java.util.List;

@Stateless
public class ProdutoFacade implements ProdutoFacadeLocal {

    @PersistenceContext(unitName = "CadastroEE-ejbPU")
    private EntityManager em;

    @Override
    public void create(Produto produto) {
        em.persist(produto);
    }

    @Override
    public void edit(Produto produto) {
        em.merge(produto);
    }
```

```
    @Override
    public void remove(Produto produto) {
        em.remove(em.merge(produto));
    }

    @Override
    public Produto find(Object id) {
        return em.find(Produto.class, id);
    }

    @Override
    public List<Produto> findAll() {
        return em.createQuery("SELECT p FROM Produto p", Produto.class)
                 .getResultList();
    }
}
```

a) Organização de um projeto corporativo no NetBeans

O projeto foi estruturado em módulos: EJB (lógica de negócio e persistência) e WAR (camada Web). O módulo EJB contém as entidades JPA (model) e as fachadas (facade/controller). O módulo WAR contém Servlets e JSPs.

b) Papel do JPA e do EJB

O JPA é responsável pelo mapeamento objeto-relacional e persistência no banco de dados. O EJB implementa as regras de negócio através de Session Beans, controlando transações e oferecendo serviços corporativos como injeção de dependência.

c) Produtividade com NetBeans

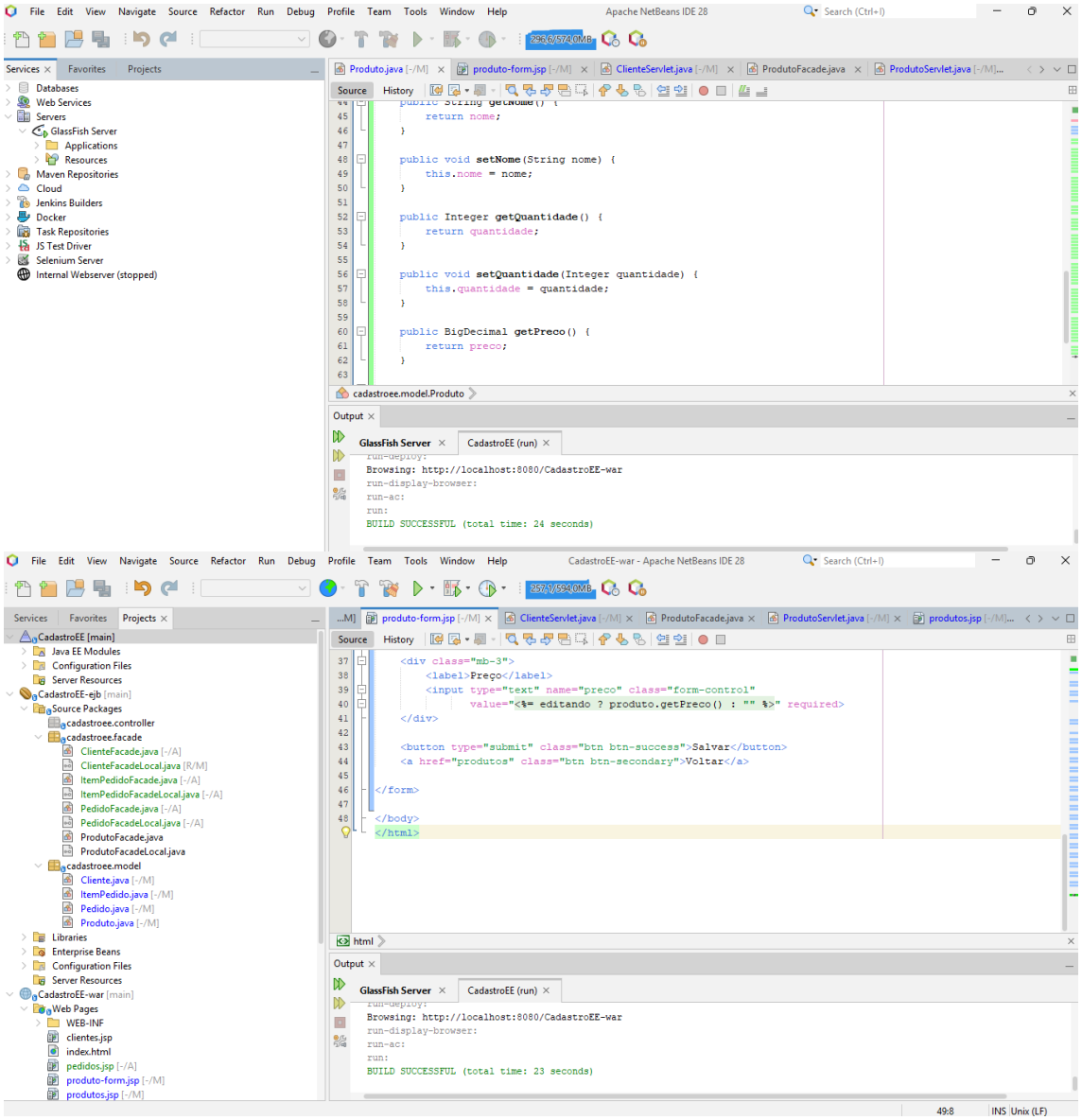
O NetBeans facilita a criação automática de entidades a partir do banco de dados, gera Session Beans (Facade) automaticamente e configura o persistence.xml, reduzindo significativamente o tempo de desenvolvimento.

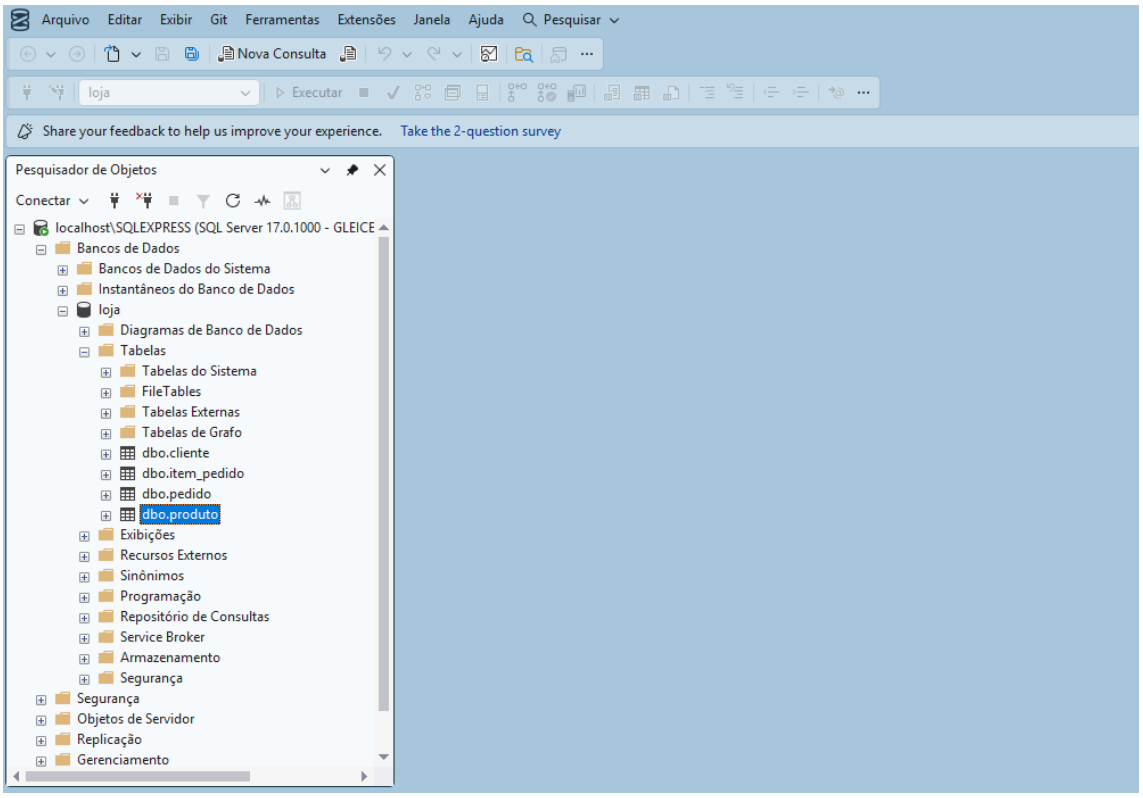
d) O que são Servlets

Servlets são classes Java que recebem requisições HTTP e geram respostas dinâmicas. No projeto, os Servlets atuam como controladores, intermediando a comunicação entre a camada Web e os EJBs.

e) Comunicação entre Servlets e EJBs

A comunicação ocorre por meio da injeção de dependência utilizando a anotação @EJB. O container do GlassFish gerencia o ciclo de vida dos EJBs e fornece instâncias aos Servlets.





2º Procedimento | Interface Cadastral com Servlet e JSPs

ProdutoServlet.java

```
package cadastroee.web;

import cadastroee.facade.ProdutoFacadeLocal;
import cadastroee.model.Produto;
import jakarta.ejb.EJB;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;
import java.io.IOException;
import java.math.BigDecimal;
import java.util.List;

@WebServlet("/produtos")
public class ProdutoServlet extends HttpServlet {

    @EJB
    private ProdutoFacadeLocal produtoFacade;

    @Override
    protected   void   doGet(HttpServletRequest   request,   HttpServletResponse
response)
            throws ServletException, IOException {

        String acao = request.getParameter("acao");

        // NOVO
        if ("novo".equals(acao)) {
            request.getRequestDispatcher("produto-form.jsp")
                   .forward(request, response);
            return;
        }

        // EDITAR
        if ("editar".equals(acao)) {
            Integer id = Integer.parseInt(request.getParameter("id"));
            Produto produto = produtoFacade.find(id);
            request.setAttribute("produto", produto);
            request.getRequestDispatcher("produto-form.jsp")
                   .forward(request, response);
            return;
        }
```

```
        // EXCLUIR
        if ("excluir".equals(acao)) {
            Integer id = Integer.parseInt(request.getParameter("id"));
            Produto produto = produtoFacade.find(id);
            produtoFacade.remove(produto);
            response.sendRedirect("produtos");
            return;
        }

        // LISTAR (default)
        List<Produto> lista = produtoFacade.findAll();
        request.setAttribute("produtos", lista);
        request.getRequestDispatcher("produtos.jsp")
               .forward(request, response);
    }

    @Override
    protected  void  doPost(HttpServletRequest  request,  HttpServletResponse
response)
            throws ServletException, IOException {

        String idParam = request.getParameter("id");
        String nome = request.getParameter("nome");
        Integer                          quantidade                          =
Integer.parseInt(request.getParameter("quantidade"));
        String precoStr = request.getParameter("preco").replace(",", ".");
        BigDecimal preco = new BigDecimal(precoStr);

        Produto produto;

        // UPDATE
        if (idParam != null && !idParam.isEmpty()) {
            produto = produtoFacade.find(Integer.parseInt(idParam));
        } else {
            produto = new Produto();
        }

        produto.setNome(nome);
        produto.setQuantidade(quantidade);
        produto.setPreco(preco);
```

```
        if (produto.getId() == null) {
            produtoFacade.create(produto);
        } else {
            produtoFacade.edit(produto);
        }

        response.sendRedirect("produtos");
    }
}
```

Produtos.jsp

```
<%@ page contentType="text/html;charset=UTF-8" %>
<%@ page import="java.util.List" %>
<%@ page import="cadastroee.model.Produto" %>


<!DOCTYPE html>
<html>
<head>
    <title>Listagem de Produtos</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container mt-4">


<h2 class="mb-4">Listagem de Produtos</h2>


<a href="produtos?acao=novo" class="btn btn-primary mb-3">
    Novo Produto
</a>


<table class="table table-striped table-bordered">
    <thead class="table-dark">
        <tr>
            <th>#</th>
            <th>Nome</th>
```

```
            <th>Quantidade</th>
            <th>Preço de Venda</th>
            <th>Opções</th>
        </tr>
    </thead>
    <tbody>
        <%
            List<Produto>          produtos          =          (List<Produto>)
request.getAttribute("produtos");
            if (produtos != null && !produtos.isEmpty()) {
                for (Produto p : produtos) {
        %>
        <tr>
            <td><%= p.getId() %></td>
            <td><%= p.getNome() %></td>
            <td><%= p.getQuantidade() %></td>
            <td><%= p.getPreco() %></td>
            <td>
                <a href="produtos?acao=editar&id=<%= p.getId() %>"
                   class="btn btn-primary btn-sm">
                   Alterar
                </a>


                <a href="produtos?acao=excluir&id=<%= p.getId() %>"
                   class="btn btn-danger btn-sm"
                   onclick="return confirm('Confirma exclusão?')">
                   Excluir
                </a>
            </td>
        </tr>
        <%
                }
            } else {
        %>
```

```
        <tr>
            <td colspan="5" class="text-center">
                Nenhum produto cadastrado.
            </td>
        </tr>
        <%
            }
        %>
    </tbody>
</table>


</body>
</html>
```

Clientes.jsp

```
<%@ page contentType="text/html;charset=UTF-8" %>
<%@ page import="java.util.List" %>
<%@ page import="cadastroee.model.Cliente" %>


<!DOCTYPE html>
<html>
<head>
    <title>Lista de Clientes</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container mt-4">


<h2>Lista de Clientes</h2>


<table class="table table-striped">
    <thead>
```

```
        <tr>
            <th>ID</th>
            <th>Nome</th>
            <th>Email</th>
        </tr>
    </thead>
    <tbody>
        <%
            List<Cliente>          clientes          =          (List<Cliente>)
request.getAttribute("clientes");
            if (clientes != null) {
                for (Cliente c : clientes) {
        %>
        <tr>
            <td><%= c.getId() %></td>
            <td><%= c.getNome() %></td>
            <td><%= c.getEmail() %></td>
        </tr>
        <%
                }
            }
        %>
    </tbody>
</table>
</body>
</html>
```

ProdutoFacadeLocal.java

```
package cadastroee.facade;
import jakarta.ejb.Local;
import java.util.List;
import cadastroee.model.Produto;

@Local
```

```
public interface ProdutoFacadeLocal {
    void create(Produto produto);
    void edit(Produto produto);
    void remove(Produto produto);
    Produto find(Object id);
    List<Produto> findAll();
}
```

a) Padrão Front Controller

O padrão Front Controller centraliza o controle das requisições em um Servlet, que decide qual ação executar e qual JSP renderizar. Na arquitetura MVC, o Servlet atua como Controller, as entidades como Model e as JSPs como View.

b) Diferenças entre Servlets e JSPs

Servlets são classes Java responsáveis pelo processamento das requisições. JSPs são páginas voltadas à apresentação, permitindo integração entre HTML e Java.

c) Redirect vs Forward

Redirect envia uma nova requisição ao cliente (response.sendRedirect). Forward (RequestDispatcher.forward) encaminha a requisição internamente no servidor. Parâmetros são enviados via URL ou formulário; atributos são objetos armazenados no request para compartilhamento interno.

3º Procedimento | Melhorando o Design da Interface

Uso do Bootstrap:

```
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
<table class="table table-striped">
```

a) Uso do Bootstrap

O Bootstrap foi utilizado via CDN para estilização das páginas JSP, aplicando classes como container, table, btn e form-control.

b) Independência estrutural

O Bootstrap separa estrutura e estilo por meio de classes CSS reutilizáveis, mantendo o HTML organizado e padronizado.

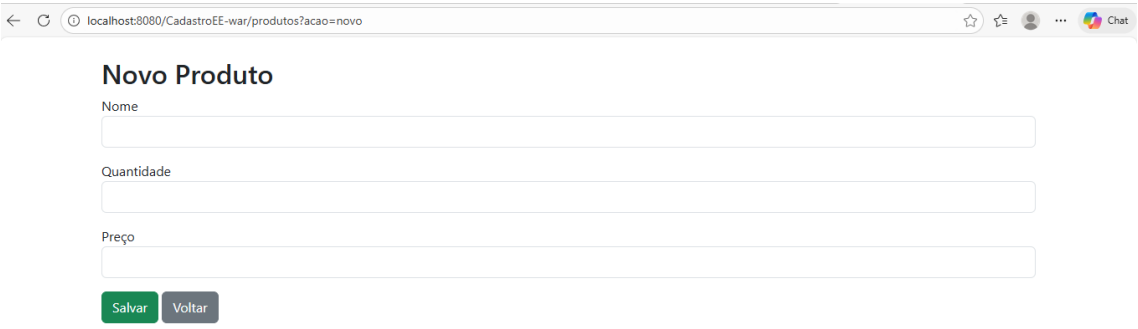
c) Responsividade

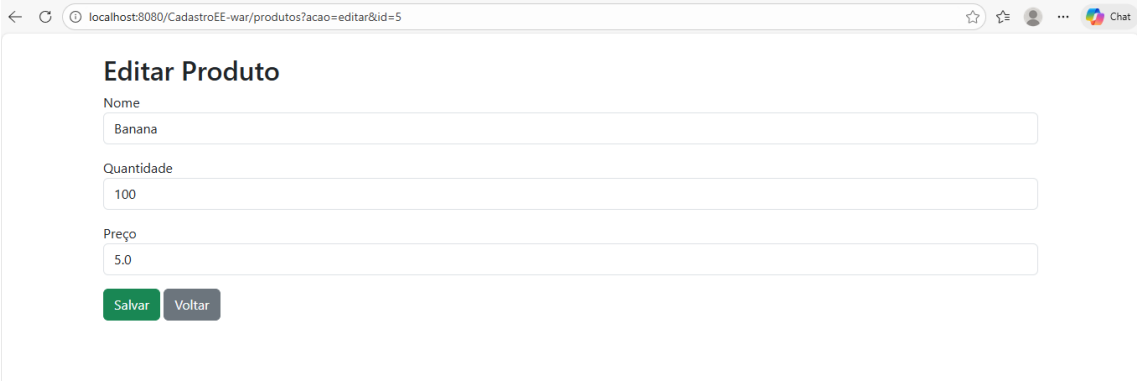
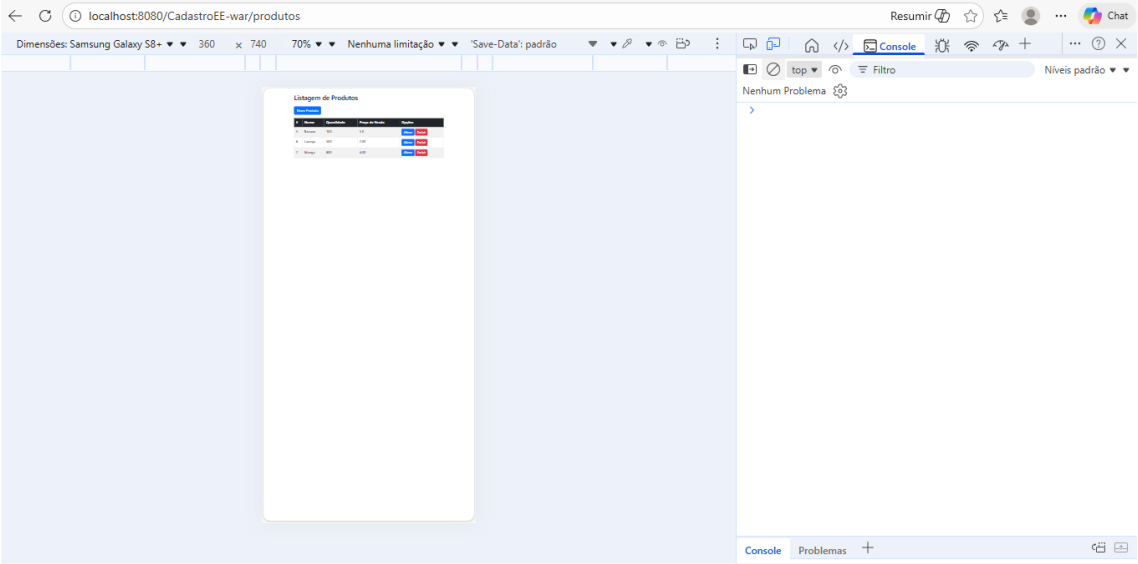
O sistema de grid do Bootstrap permite adaptação automática da interface a diferentes tamanhos de tela, garantindo responsividade.

Conclusão

A prática permitiu compreender a organização de aplicações corporativas Java EE, a integração entre JPA, EJB, Servlets e JSP, além da importância da separação em camadas. Também foi possível aplicar conceito de arquitetura MVC e melhorar a interface com Bootstrap.

Evidências:





Responsividade